

LSTM & GRU Gates, Worked Out by Hand

One timestep through every gate, with real sigmoids and tanh

A complete numerical walk-through of a single LSTM timestep. We pick the smallest cell that still exercises every mechanism — one hidden unit, one input feature — fix concrete tiny weights, and push one input through the *forget*, *input*, *output* and *candidate* gates, then update the cell state and read out the hidden state. Nothing is hand-waved: every sigmoid and tanh below was produced by the NumPy/PyTorch reference program in Appendix A, and the final (h_t, c_t) is checked against `torch.nn.LSTMCell` to the sixth decimal.

What this document does. It takes one tiny LSTM cell with previous states $h_{t-1} = 0$, $c_{t-1} = 1$ and input $x_t = 1$, chooses weights so the four pre-activations land on the clean points $\{+2, -2, +2, +1\}$, and then computes the exact gate openings f, i, o, g , the additive cell update $c_t = f c_{t-1} + i g$, and the gated read-out $h_t = o \tanh(c_t)$. It closes with the GRU's two-gate variant on the same footing.

How to read this. Each gate is derived on its own, with a *Mini-example* or *Intuition* box isolating its role. The sigmoid is a soft valve in $[0, 1]$ (how much to let through); tanh in $[-1, 1]$ supplies the signed content. Read this to understand *why* the cell state is an additive conveyor belt — the structural reason LSTMs fight vanishing gradients where plain RNNs fail.

Concepts covered, in order: the two states (cell c_t vs hidden h_t) · the shared gate template $\sigma(W[h_{t-1}, x_t] + b)$ · forget gate (keep/erase memory) · input gate & candidate (write new) · the additive cell update · output gate (read out) · the full assembled timestep · the GRU as a 2-gate merge of cell and hidden · gate saturation and the forget-bias trick.

1 The setup: a cell small enough to compute by hand

A real LSTM layer carries a hidden vector of size 64 or 256 and a weight matrix of shape $(4H, H+I)$. You cannot follow that on paper, so we shrink to the smallest cell that still has every gate: hidden size $H = 1$, input size $I = 1$. Each "matrix" is then a single number, and each gate is one scalar in $[0, 1]$.

Quantity	Symbol	Value here
Previous hidden state	h_{t-1}	0
Previous cell state	c_{t-1}	1
Current input	x_t	1
Hidden / input size	H, I	1, 1
Gate nonlinearity	σ	logistic sigmoid, range $(0, 1)$
Content nonlinearity	$\tanh$	range $(-1, 1)$

Two states, two jobs. The course frames the LSTM as carrying a *cell state* c_t — long-term memory, the “conveyor belt” — and a *hidden state* h_t — short-term working memory and the layer’s output at this step. Three gates decide what to *forget* from the belt, what to *write* onto it, and what to *read out* of it.

Every gate uses the *same* template: stack the previous hidden state and the current input into one vector $[h_{t-1}, x_t]$, apply an affine map, then squash:

$$\text{gate} = \sigma(W[h_{t-1}, x_t] + b) = \sigma(W_h h_{t-1} + W_x x_t + b)$$

With $h_{t-1} = 0$ the W_h term vanishes this step, so every pre-activation collapses to $W_x x_t + b$. We pick the weights below to make those land on round numbers.

Gate	W_h	W_x	b	pre-activation $z = W_x x_t + b$
forget f	0.5	1.0	1.0	$1(1) + 1.0 = +2$
input i	0.5	-1.0	-1.0	$-1(1) - 1.0 = -2$
output o	-0.5	1.5	0.5	$1.5(1) + 0.5 = +2$
candidate g	0.5	0.5	0.5	$0.5(1) + 0.5 = +1$

Intuition

A gate is a **differentiable valve**. The sigmoid maps any real pre-activation into $[0, 1]$: think of it as “what fraction do I let through?” — 0 is shut, 1 is wide open, 0.5 is half. Because it is smooth, the network can *learn* how far to open each valve by gradient descent, instead of making a hard yes/no decision. The tanh is a different organ: it produces *signed content* in $[-1, 1]$ — the actual value you might write to memory — which a sigmoid valve then admits or blocks.

2 Step 0 — The two squashing functions, evaluated

Everything downstream is built from two functions. Compute the exact values we will reuse, in `float64`, rounded to three decimals for display:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma(0) = 0.500, \quad \sigma(+2) = 0.881, \quad \sigma(-2) = 0.119.$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh(1) = 0.762, \quad \tanh(0.5) = 0.462.$$

Note the symmetry $\sigma(-z) = 1 - \sigma(z)$: that is why $\sigma(+2) = 0.881$ and $\sigma(-2) = 0.119$ sum to exactly 1. Both functions are flat far from the origin — a fact that will matter for gradients in Section 8.

3 Step 1 — Forget gate: how much old memory to keep

The forget gate looks at $[h_{t-1}, x_t]$ and outputs, per memory cell, a number in $[0, 1]$: the fraction of the *old* cell state to retain. With our forget weights:

$$z_f = W_{f,h} h_{t-1} + W_{f,x} x_t + b_f = 0.5(0) + 1.0(1) + 1.0 = 2, \quad \boxed{f = \sigma(2) = 0.881}.$$

So 88.1% of c_{t-1} survives to the next step; 11.9% is erased. With $c_{t-1} = 1$ the kept amount is

$$f \cdot c_{t-1} = 0.881 \times 1 = 0.881.$$

Mini-example — this operation in isolation

The forget gate *is* a scalar multiplier on memory. Three openings on the same $c_{t-1} = 1$:

$$f = \sigma(2) = 0.881 \Rightarrow \text{keep } 0.881, \quad f = \sigma(0) = 0.500 \Rightarrow \text{keep } 0.500, \quad f = \sigma(-2) = 0.119 \Rightarrow \text{keep } 0.119.$$

$f \rightarrow 1$ means “remember everything” (the belt passes untouched); $f \rightarrow 0$ means “wipe this cell.” A plain RNN has no such knob — it must overwrite its state every step, which is exactly why it forgets.

4 Step 2 — Input gate and candidate: what new value to write

Writing is two decisions, computed in parallel. The **candidate** g proposes a signed value to add (via $\tanh$); the **input gate** i decides how much of that proposal to actually admit (via sigmoid).

$$z_g = 0.5(0) + 0.5(1) + 0.5 = 1, \quad g = \tanh(1) = 0.762 \quad (\text{the proposed content}).$$

$$z_i = 0.5(0) - 1.0(1) - 1.0 = -2, \quad i = \sigma(-2) = 0.119 \quad (\text{the write valve, nearly shut}).$$

The amount actually written this step is the product

$$i \cdot g = 0.119 \times 0.762 = 0.091.$$

The valve is almost closed ($i = 0.119$), so even though the candidate proposes a fairly large $+0.762$, only 0.091 lands on the belt.

Intuition

Splitting “write” into a *gate* and a *candidate* decouples *whether* to write from *what* to write. The candidate $g \in [-1, 1]$ can be strongly positive or negative — new information of either sign — while the input gate $i \in [0, 1]$ independently controls the dose. The cell can therefore hold a value steady (shut the input gate) while still being *ready* to overwrite the moment the gate opens.

5 Step 3 — The cell update: an additive conveyor belt

Now combine the two flows. The new cell state is the kept memory plus the gated write:

$$c_t = f \cdot c_{t-1} + i \cdot g$$

$$c_t = \underbrace{0.881}_{f \cdot c_{t-1}} + \underbrace{0.091}_{i \cdot g} = 0.972.$$

That is the entire long-term memory mechanism: scale the old value, add a gated new value. Crucially the operation is **additive**, not a fresh nonlinear overwrite.

Intuition

The additive path $c_t = f c_{t-1} + i g$ is the structural heart of the LSTM. Trace the gradient of c_t with respect to c_{t-1} : it is just f . Over T steps the memory gradient multiplies a chain of forget gates, $\prod_t f_t$, with *no* repeated squashing nonlinearity in the way. If the forget gates stay near 1, that product stays near 1 and the gradient survives across long spans. A vanilla RNN instead multiplies a chain of $\tanh'$ terms (each < 1), so its gradient decays geometrically — the vanishing-gradient problem the LSTM was invented to cure.

6 Step 4 — Output gate: reading the memory out as h_t

The hidden state is a *gated view* of the cell. First squash the cell through $\tanh$ to bound it, then admit a fraction through the output gate:

$$z_o = -0.5(0) + 1.5(1) + 0.5 = 2, \quad o = \sigma(2) = 0.881.$$

$$\tanh(c_t) = \tanh(0.972) = 0.749, \quad h_t = o \cdot \tanh(c_t) = 0.881 \times 0.749 = 0.660.$$

So although the belt now holds $c_t = 0.972$, the cell only *exposes* $h_t = 0.660$ to the rest of the network. The remaining memory stays internal, available for later steps but hidden from this one’s output.

Watch out

The cell state $c_t = 0.972$ and the hidden state $h_t = 0.660$ are *different* numbers and play different roles. c_t is private long-term memory passed forward on the belt; h_t is the public output of this timestep *and* the input to next step’s gates. Confusing them — e.g. feeding c_t where h_t is expected — silently breaks the recurrence. `nn.LSTM` returns both; keep the tuple `(h_n, c_n)` straight.

7 Step 5 — The whole timestep, assembled

Putting the five steps in order, the complete forward pass for this cell is:

Stage	Formula	Value	Meaning
forget gate	$f = \sigma(z_f)$	0.881	keep 88.1% of c_{t-1}
input gate	$i = \sigma(z_i)$	0.119	write valve nearly shut
candidate	$g = \tanh(z_g)$	0.762	proposed new content
output gate	$o = \sigma(z_o)$	0.881	expose 88.1% of memory
kept memory	$f c_{t-1}$	0.881	old value, scaled
new write	$i g$	0.091	gated candidate
cell state	$c_t = f c_{t-1} + i g$	0.972	updated belt
squashed cell	$\tanh(c_t)$	0.749	bounded to $(-1, 1)$
hidden state	$h_t = o \tanh(c_t)$	0.660	this step's output

The new state pair $(h_t, c_t) = (0.660, 0.972)$ is handed to the next timestep, where h_t re-enters every gate's $[h_{t-1}, x_t]$ vector and c_t rides the belt forward. Appendix A reproduces exactly these two numbers with `torch.nn.LSTMCell`.

8 Step 6 — The GRU: same idea, two gates, one state

The GRU keeps the gating idea but drops the separate cell state: there is only h_t , and just two gates. The **update gate** z plays the combined role of forget-and- input; the **reset gate** r controls how much past state feeds the candidate. We work a fresh tiny example with $h_{t-1} = 0.5$, $x_t = 1$.

$$z = \sigma(z_z), \quad z_z = W_{z,x}x_t + b_z = 1(1) + 1 = 2 \Rightarrow z = \sigma(2) = 0.881.$$

$$r = \sigma(z_r), \quad z_r = W_{r,x}x_t + b_r = -1(1) - 1 = -2 \Rightarrow r = \sigma(-2) = 0.119.$$

The reset gate scales the past state *before* it enters the candidate:

$$r \cdot h_{t-1} = 0.119 \times 0.5 = 0.060, \quad \tilde{h} = \tanh(W_x x_t + U(r h_{t-1})) = \tanh(0.5) = 0.462.$$

Finally the update gate *linearly interpolates* between the old state and the candidate:

$$h_t = (1 - z) h_{t-1} + z \tilde{h}$$

$$h_t = \underbrace{0.119 \times 0.5}_{0.060} + \underbrace{0.881 \times 0.462}_{0.407} = 0.467.$$

Intuition

The GRU's update gate is a single dial that does the LSTM's forget *and* input work at once: $h_t = (1 - z)h_{t-1} + z\tilde{h}$ means “keep fraction $(1 - z)$ of the old, take fraction z of the new.” Because the two halves share one z , what you don't overwrite you automatically retain — a built-in conservation that gives gradients a clean linear path, much like the LSTM's additive belt, but with fewer parameters (2 gates vs. 3, no separate c_t).

When this is the right choice

Reach for an LSTM or GRU over a Transformer when: the data is **small** (under $\sim 10k$ examples — the recurrent inductive bias generalises where attention overfits); inference is **strictly streaming** / **causal** (one token at a time, with constant memory, no re-attending the whole context); or sequences are **long but not enormous**, where attention's $O(T^2)$ memory hurts but a linear-memory recurrence stays cheap. GRUs in particular export cleanly to ONNX/T-FLite for edge deployment.

9 Step 7 — Saturation, and the forget-bias trick

Gates are valves, but valves can rust open or shut. When a pre-activation is large in magnitude the sigmoid *saturates*: $\sigma(z) \rightarrow 1$ or 0 and its slope $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \rightarrow 0$. A saturated gate passes *no gradient*, so it stops learning. At our values $\sigma'(2) = 0.881(0.119) = 0.105$ — already small.

The classic fix is to **initialise the forget-gate bias to about +1** so cells *remember by default*. Watch the bias move the kept fraction:

$$b_f = +1 : z_f = 2, f = 0.881; \quad b_f = 0 : z_f = 1, f = 0.731; \quad b_f = -1 : z_f = 0, f = 0.500.$$

Watch out

With $b_f = 0$ a freshly-initialised LSTM starts at $f \approx 0.5$, halving its memory *every* step — after 10 steps only $0.5^{10} \approx 0.001$ survives, and the gradient vanishes before training even gets going. Setting $b_f \approx +1$ starts cells near $f = 0.881$ (keep $\sim 88\%$ per step), so memory and gradient persist by default and the network *learns* when to forget rather than starting amnesiac. PyTorch leaves this to you — many recipes set it explicitly.

10 LSTM vs. GRU, at a glance

	LSTM (this doc)	GRU
States carried	c_t and h_t	h_t only
Gates	f, i, o (3)	z, r (2)
Candidate	$g = \tanh(\cdot)$	$\tilde{h} = \tanh(\cdot)$
Memory update	$c_t = f c_{t-1} + i g$	$h_t = (1-z)h_{t-1} + z\tilde{h}$
Read-out	$h_t = o \tanh(c_t)$	h_t is the state
Params (per unit)	more (~ 4 sets)	fewer (~ 3 sets)
Worked result here	$c_t=0.972, h_t=0.660$	$h_t=0.467$

Both share the same DNA: differentiable $[0, 1]$ valves guarding an (almost) linear path for memory, so gradients can travel across many timesteps without vanishing.

11 Equation cheat-sheet

Gate template	$\text{gate} = \sigma(W_h h_{t-1} + W_x x_t + b)$
Forget gate	$f = \sigma(W_f[h_{t-1}, x_t] + b_f)$
Input gate	$i = \sigma(W_i[h_{t-1}, x_t] + b_i)$
Candidate	$g = \tanh(W_g[h_{t-1}, x_t] + b_g)$
Output gate	$o = \sigma(W_o[h_{t-1}, x_t] + b_o)$
Cell update	$c_t = f \odot c_{t-1} + i \odot g$
Hidden read-out	$h_t = o \odot \tanh(c_t)$
Sigmoid derivative	$\sigma'(z) = \sigma(z) (1 - \sigma(z))$
GRU update / reset	$z = \sigma(\cdot), r = \sigma(\cdot)$
GRU candidate	$\tilde{h} = \tanh(W_x x_t + U(r \odot h_{t-1}))$
GRU state	$h_t = (1 - z) \odot h_{t-1} + z \odot \tilde{h}$

Appendix A Reference implementation

Every number above is reproducible from the following program. The hand-derived (h_t, c_t) is checked against `torch.nn.LSTMCell`, whose gate order in the weight tensors is i, f, g, o .

```
import numpy as np, torch

sig = lambda z: 1/(1+np.exp(-z))

# ---- one LSTM timestep, hidden = input = 1 ----
h_prev, c_prev, x = 0.0, 1.0, 1.0

# weights chosen so pre-activations land on +2,-2,+2,+1
Wf_h,Wf_x,bf = 0.5, 1.0, 1.0      # zf = +2
Wi_h,Wi_x,bi = 0.5,-1.0,-1.0     # zi = -2
Wo_h,Wo_x,bo = -0.5, 1.5, 0.5     # zo = +2
Wg_h,Wg_x,bg = 0.5, 0.5, 0.5     # zg = +1

zf = Wf_h*h_prev + Wf_x*x + bf
zi = Wi_h*h_prev + Wi_x*x + bi
zo = Wo_h*h_prev + Wo_x*x + bo
zg = Wg_h*h_prev + Wg_x*x + bg

f, i, o = sig(zf), sig(zi), sig(zo)
g      = np.tanh(zg)
c_t = f*c_prev + i*g      # 0.972
h_t = o*np.tanh(c_t)      # 0.660
print(f"f={f:.3f} i={i:.3f} o={o:.3f} g={g:.3f}")
print(f"c_t={c_t:.3f} h_t={h_t:.3f}")

# ---- check against torch.nn.LSTMCell (gate order i,f,g,o) ----
cell = torch.nn.LSTMCell(1, 1)
with torch.no_grad():
    cell.weight_ih.copy_(torch.tensor([[Wi_x],[Wf_x],[Wg_x],[Wo_x]]))
    cell.weight_hh.copy_(torch.tensor([[Wi_h],[Wf_h],[Wg_h],[Wo_h]]))
    cell.bias_ih.copy_(torch.tensor([bi, bf, bg, bo]))
    cell.bias_hh.copy_(torch.zeros(4))
    h1, c1 = cell(torch.tensor([[x]]),
                  (torch.tensor([[h_prev]]), torch.tensor([[c_prev]])))
print("torch:", round(h1.item(),3), round(c1.item(),3)) # 0.660 0.972

# ---- GRU timestep, hidden = 1 ----
hg, xg = 0.5, 1.0
z = sig(1*xg + 1)      # update gate sigma(+2) = 0.881
r = sig(-1*xg - 1)     # reset gate sigma(-2) = 0.119
htilde = np.tanh(0.440399*xg + 1*(r*hg)) # tanh(0.5) = 0.462
h_new = (1-z)*hg + z*htilde # 0.467
print(f"z={z:.3f} r={r:.3f} htilde={htilde:.3f} h_new={h_new:.3f}")
```

— end of document —